

BEE 4750 Homework 1: Introduction to Using Julia

Name: William Dignam

ID: wed45, 5493731

Due Date

Thursday, 9/03/26, 9:00pm

Overview

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

Activating project at `c:\Users\billy\OneDrive\Desktop\BEE 4750\HW1`
⌈ Warning: The project dependencies or compat requirements have changed since the manifest was last resolved.
| It is recommended to `Pkg.resolve()` or consider `Pkg.update()` if necessary.
└ @ Pkg.API C:\Users\billy\julia\juliaup\julia-1.12.7+0.x64.w64.mingw32\share\julia\stdlib\v1.12\Pkg\src\API.jl:1322
Precompiling packages...
 x MbedTLS
 x HTTP
0 dependencies successfully precompiled in 51 seconds. 258 already precompiled.
209 dependencies precompiled but different versions are currently loaded (AbstractTrees, Adapt, Adapt → AdaptSparseArraysExt, Adapt → AdaptStaticArraysExt, AliasTables, ArgTools, ArnoldiMethod, AxisAlgorithms, Base64, Bzip2_jll, CSV, Cairo_jll, ChainRulesCore, ChainRulesCore → ChainRulesCoreSparseArraysExt, CodecZlib, ColorSchemes, ColorTypes, ColorVectorSpace, ColorVectorSpace → SpecialFunctionsExt, Colors, Compat, Compat → CompatLinearAlgebraExt, CompilerSupportLibraries_jll, Conda, Crayons, DataAPI, DataFrames, DataStructures, DataValueInterfaces, Dates, Distributed, Distributions, Distributions → DistributionsChainRulesCoreExt, DocStringExtensions, Downloads, EarCut_jll, Expat_jll, Extents, FFMPEG, FFMPEG_jll, FilePathsBase, FilePathsBase → FilePathsBaseMmapExt, FillArrays, FillArrays → FillArraysPDMatsExt,

FillArrays → FillArraysSparseArraysExt, FillArrays →
 FillArraysStatisticsExt, FixedPointNumbers, Fontconfig_jll,
 FreeType2_jll, FriBidi_jll, Future, GLFW_jll, GR, GR_jll,
 GeometryBasics, GeometryTypes, GettextRuntime_jll, Glib_jll,
 GraphRecipes, Graphite2_jll, Graphs, HarfBuzz_jll,
 HypergeometricFunctions, IJulia, Inflate, InlineStrings, InlineStrings
 → ParsersExt, InteractiveUtils, Interpolations, InvertedIndices,
 IrrationalConstants, IterTools, IteratorInterfaceExtensions, JLFzf,
 JLLWrappers, JSON, JpegTurbo_jll, LAME_jll, LERC_jll, LZ0_jll,
 LaTeXStrings, LibCURL, LibCURL_jll, LibGit2, LibGit2_jll, LibSSH2_jll,
 Libffi_jll, Libglvnd_jll, Libiconv_jll, Libtiff_jll, Libuuid_jll,
 LogExpFunctions, LogExpFunctions → LogExpFunctionsChainRulesCoreExt,
 Logging, MacroTools, Markdown, Measures, Missings, Mmap,
 MozillaCACerts_jll, NaNMath, NetworkLayout, NetworkLayout →
 NetworkLayoutGraphsExt, NetworkOptions, OffsetArrays, OffsetArrays →
 OffsetArraysAdaptExt, Ogg_jll, OpenLibm_jll, OpenSSL_jll,
 OpenSpecFun_jll, Opus_jll, OrderedCollections, PCRE2_jll, PDMats,
 Parsers, Pixman_jll, Pkg, Pkg → REPLExt, PlotThemes, PlotUtils, Plots,
 Plots → GeometryBasicsExt, Plots → IJuliaExt, PooledArrays,
 PrecompileTools, Preferences, PrettyTables, Printf, PtrArrays,
 Qt6Base_jll, QuadGK, REPL, Ratios, Ratios →
 RatiosFixedPointNumbersExt, RecipesBase, RecipesPipeline, Reexport,
 RelocatableFolders, Requires, Rmath, Rmath_jll, Scratch,
 SentinelArrays, Serialization, SharedArrays, Showoff, SimpleTraits,
 SoftGlobalScope, SortingAlgorithms, SparseArrays, SpecialFunctions,
 SpecialFunctions → SpecialFunctionsChainRulesCoreExt, StableRNGs,
 StaticArrays, StaticArrays → StaticArraysChainRulesCoreExt,
 StaticArrays → StaticArraysStatisticsExt, StaticArraysCore,
 Statistics, Statistics → SparseArraysExt, StatsAPI, StatsBase,
 StatsFuns, StatsFuns → StatsFunsChainRulesCoreExt, StringManipulation,
 StyledStrings, SuiteSparse, SuiteSparse_jll, TOML, TableTraits,
 Tables, Tar, TensorCore, TranscodingStreams, UUIDs, Unicode,
 UnicodeFun, Unzip, VersionParsing, Vulkan_Loader_jll, WeakRefStrings,
 WoodburyMatrices, WorkerUtilities, XZ_jll, Xorg_libSM_jll,
 Xorg_libXext_jll, Xorg_libXrender_jll, Xorg_libxcb_jll,
 Xorg_xcb_util_cursor_jll, Xorg_xcb_util_image_jll,
 Xorg_xcb_util_keysyms_jll, Xorg_xcb_util_renderutil_jll,
 Xorg_xcb_util_wm_jll, ZMQ, ZeroMQ_jll, Zlib_jll, Zstd_jll, fzf_jll,
 libaom_jll, libass_jll, libfdk_aac_jll, libinput_jll, libpng_jll,
 libsodium_jll, libvorbis_jll, nghttp2_jll, p7zip_jll, x264_jll,
 x265_jll and xkbcommon_jll). Restart julia to access the new versions.
 Otherwise, 45 dependents of these packages may trigger further
 precompilation to work with the unexpected versions.

Standard Julia practice is to load all needed packages at the top of a file. If you need to load any additional packages in any assignments beyond those which are loaded by default, feel free to add a `using` statement, though [you may need to install the package](#).

```

using Random # allows for random seed generation
using Plots # basic plotting: can also install and use other packages
using Graphs # for making graphs and networks
using GraphRecipes # basic graph generation
using LaTeXStrings # allows for LaTeX formatting in plots
using Distributions # sampling and fitting of probability
distributions
using CSV # file I/O for CSV files
using DataFrames # data structure for tabular data

# this sets a random seed, which ensures reproducibility of random
number generation. You should always set a seed when working with
random numbers.
Random.seed!(1)

TaskLocalRNG()

```

Problems (Total: 50 Points)

Problem 1 (10)

Systems Modeling:

Three factories are discharging a chemical called Chlororadiated Ureadicarboxyl (or CRUD) into the Riley River. The river inflow has a volume of $500,000 \text{ m}^3/\text{d}$ and a CRUD concentration of 0.2 mg/L . The river velocity is 25 km/d .

- Factory 1 discharges $100,000 \text{ m}^3/$ of effluent with a CRUD concentration of 10 mg/L ;
- Factory 2 discharges $60,000 \text{ m}^3/$ of effluent with a CRUD concentration of 20 mg/L ;
- Factory 3 discharges $200,000 \text{ m}^3/$ of effluent with a CRUD concentration of 8 mg/L .

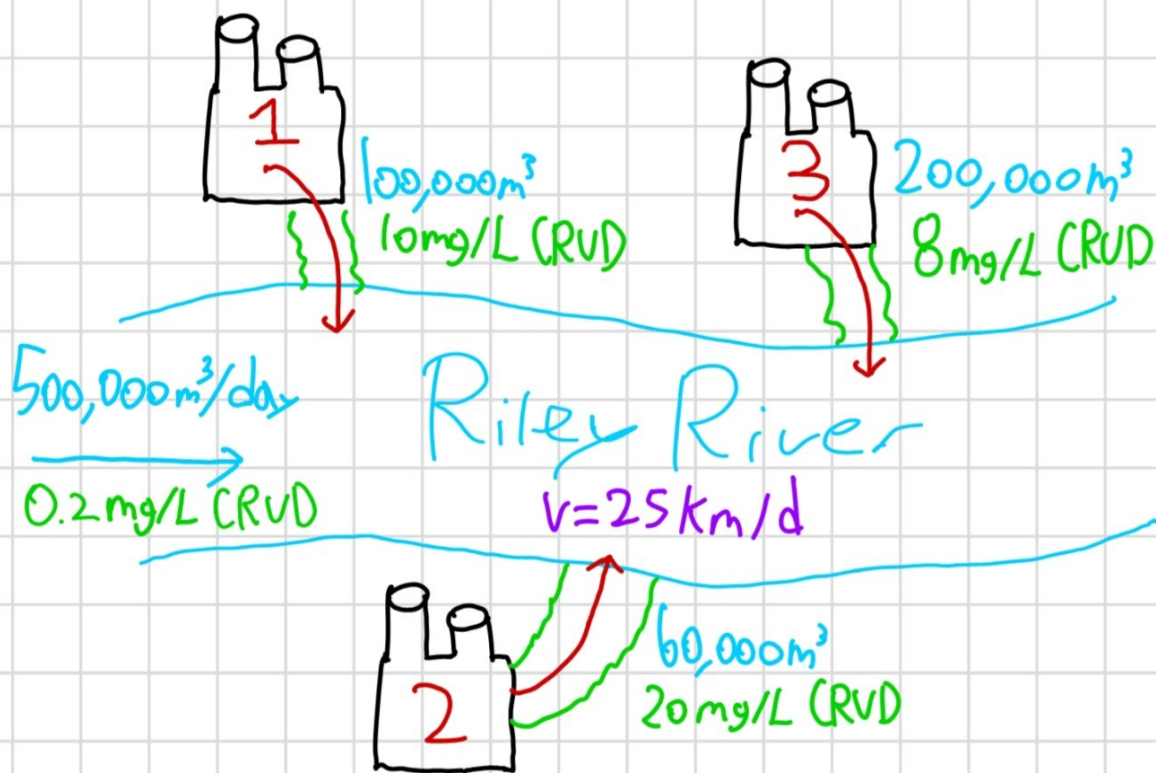
CRUD decays in the river with a first order reaction rate $k = 0.45 \text{ d}^{-1}$.

Environmental authorities have taken samples downriver of the three factories and found that the CRUD concentration exceeds the regulatory limit of 1 mg/L . CRUD can be removed at the source discharge at a cost of $50 E^2$ per 1000 m^3 treated, where E is the efficiency of removal (the fraction of CRUD removed).

Problem 1.1 (2)

Draw a diagram of the river and factory system. Include definitions for any notation you will use.

1.1



Problem 1.2 (5)

Write down a set of conditions on the removal efficiencies E_1 , E_2 , and E_3 (where the subscript corresponds to the factory where the effluent is being treated) to guarantee compliance.

Let E_1 , E_2 , and E_3 denote the fractions of CRUD removed from the effluent of Factories 1, 2, and 3, respectively.

Because $1 \text{ mg/L} = 1 \text{ g/m}^3$, the incoming river carries

$$(500,000)(0.2) = 100,000 \text{ g/day} = 100 \text{ kg/day}$$

of CRUD. The untreated factory loads are

$$L_1 = (100,000)(10) = 1,000 \text{ kg/day},$$

$$L_2 = (60,000)(20) = 1,200 \text{ kg/day},$$

and

$$L_3 = (200,000)(8) = 1,600 \text{ kg/day}.$$

Immediately downstream of Factory 1, the concentration must satisfy

$$\frac{100+1,000(1-E_1)}{600,000} \left(1,000 \frac{L}{m^3} \right) \leq 1 \text{ mg/L}.$$

This simplifies to

$$E_1 \geq 0.50.$$

Immediately downstream of Factory 2, compliance requires

$$100+1,000(1-E_1)+1,200(1-E_2) \leq 660 \text{ kg/day},$$

or equivalently,

$$1,000 E_1 + 1,200 E_2 \geq 1,640.$$

Immediately downstream of Factory 3, compliance requires

$$100+1,000(1-E_1)+1,200(1-E_2)+1,600(1-E_3) \leq 860 \text{ kg/day},$$

or equivalently,

$$1,000 E_1 + 1,200 E_2 + 1,600 E_3 \geq 3,040.$$

Therefore, a sufficient set of conditions guaranteeing compliance is

$$\begin{aligned} 0 & \leq E_1, E_2, E_3 \leq 1, \\ E_1 & \geq 0.50, \\ 1,000 E_1 + 1,200 E_2 & \geq 1,640, \\ 1,000 E_1 + 1,200 E_2 + 1,600 E_3 & \geq 3,040. \end{aligned}$$

These conditions neglect first-order decay between factories. Since decay only removes CRUD from the river, including it would relax the required removal efficiencies.

Problem 1.3 (3)

Without finding an "optimal" solution, reason about which factories ought to treat their effluent at a higher level than the others. This can draw on considerations such as cost, the sequencing of discharges, and the amount of CRUD discharged by each factory, or others.

Considering treatment cost and CRUD discharged, Factory 2 should generally treat its effluent at the highest level, followed by Factory 1 and then Factory 3. The daily treatment costs for the three factories are

$$C_1 = 5,000 E_1^2, C_2 = 3,000 E_2^2, C_3 = 10,000 E_3^2.$$

(considering $50E^2$ for every $1000m^3$)

The quantities of CRUD removed are:

$$R_1 = 1,000 E_1, R_2 = 1,200 E_2, R_3 = 1,600 E_3.$$

The marginal treatment cost per unit of CRUD removed is therefore (as a rate):

$$\frac{dC_1/dE_1}{dR_1/dE_1} = 10E_1, \frac{dC_2/dE_2}{dR_2/dE_2} = 5E_2, \frac{dC_3/dE_3}{dR_3/dE_3} = 12.5E_3.$$

At equal removal efficiencies, Factory 2 has the lowest marginal cost per unit of CRUD removed, followed by Factory 1 and then Factory 3. Therefore, Factory 2 should generally treat its effluent at a higher level. However, because each cost function is quadratic, additional treatment does get more expensive, which is something to be considered in treatment for all three factories.

Problem 2 (6)

Systems Perturbations.

Consider the perturbed phosphorous cycle example from Lecture 2. Now suppose that the initial perturbation at $t=0$ is a 10% reduction in the rate constant for the conversion of phosphorous from organic to inorganic form (through an inhibition of phosphatase activity due to e.g. acidic rain fall). When the new steady state is reached, how much phosphorous will be in each compartment?

The phosphorus model is:

$$X_1 = \beta X_1 X_2 - \gamma X_1,$$

$$X_2 = \alpha X_3 - \beta X_1 X_2,$$

$$X_3 = \gamma X_1 - \alpha X_3.$$

The original steady-state concentrations are

$$\bar{X}_1 = 0.2, \bar{X}_2 = 0.1, \bar{X}_3 = 1.0 \text{ all in } \mu\text{mol/L}.$$

The residence time of phosphorus in living biomass is four days. Because the outflow from living biomass is γX_1 ,

$$\tau_1 = \frac{X_1}{\gamma X_1} = \frac{1}{\gamma} = 4,$$

so

$$\gamma = 0.25 \text{ day}^{-1}.$$

At the original steady state (and considering $t=0$ initially),

$$\beta \bar{X}_1 \bar{X}_2 = \gamma \bar{X}_1,$$

which gives

$$\beta = \frac{\gamma}{\bar{X}_2} = \frac{0.25}{0.1} = 2.5 \text{ L } \mu\text{mol}^{-1} \text{ day}^{-1}.$$

Also,

$$\alpha \bar{X}_3 = \gamma \bar{X}_1,$$

so

$$\alpha = \frac{\gamma \bar{X}_1}{\bar{X}_3} = \frac{(0.25)(0.2)}{1.0} = 0.05 \text{ d a y}^{-1}.$$

A 10% reduction in α gives

$$\alpha_{new} = 0.90 \alpha = 0.045 \text{ d a y}^{-1}.$$

At the new positive steady state,

$$0 = X_1(\beta X_2 - \gamma),$$

so

$$X_2^i = \frac{\gamma}{\beta} = \frac{0.25}{2.5} = 0.1 \mu\text{mol/L}.$$

The balance between living and dead organic phosphorus gives

$$\alpha_{new} X_3^i = \gamma X_1^i,$$

and therefore

$$X_3^i = \frac{\gamma}{\alpha_{new}} X_1^i = \frac{0.25}{0.045} X_1^i = \frac{50}{9} X_1^i.$$

Because changing the rate constant does not add or remove phosphorus, total phosphorus remains constant:

$$X_1^i + X_2^i + X_3^i = 0.2 + 0.1 + 1.0 = 1.3.$$

Substituting $X_2^i = 0.1$ and $X_3^i = \frac{50}{9} X_1^i$ gives

$$X_1^i + \frac{50}{9} X_1^i = 1.2.$$

Therefore,

$$X_1^i \approx 0.1831 \mu\text{mol/L},$$

$$X_2^i = 0.1000 \mu\text{mol/L},$$

and

$$X_3^i \approx 1.0169 \mu\text{mol/L}.$$

Phosphorus accumulates slightly in dead organic matter, while phosphorus in living biomass decreases. The inorganic phosphorus concentration remains unchanged at the positive steady state because it is determined by the ratio γ/β , both of which don't change

Problem 3 (9)

Debugging Code:

The following subproblems all involve code snippets that require debugging. You are encouraged to use online resources (*e.g.* Julia documentation and forums, Stack Overflow, Reddit, etc) to help find diagnose error messages and find solutions, but make sure that you clearly document which resources you used and how you used them.

Problem 3.1 (3)

You've been tasked with writing code to identify the minimum value in an array. You cannot use a predefined function. Your colleague suggested the function below, but it does not return the minimum value.

```
function minimum(array)
    # initialize the minimum value counter
    min_value = array[1]
    # update minimum values
    for i in 1:length(array)
        if array[i] < min_value
            min_value = array[i]
        end
    end
    # return found minimum
    return min_value
end

array_values = [89, 90, 95, 100, 100, 78, 99, 98, 100, 95]
@show minimum(array_values);

minimum(array_values) = 78
```

What is the logical flaw in the code? Fix it (describe what you changed rather than reproducing the entire code snippet) and show that your fixed code solves the problem.

Logical flaw / issue: All of the array values are positive numbers above the value "0". Therefore, "array[i] < min_value" will always be false, and the function will always return 0 even though 0 isn't even in the array. The solution is to make "min_value" equal to the first value in the array. The fixed code is above and returns "78" as the minimum value.

Problem 3.2 (3)

Your team wants to know the expected payout of an old Italian dice game called *passadieci* (which was analyzed by Galileo as one of the first examples of a rigorous study of probability). The goal of *passadieci* is to get at least an 11 from rolling three fair, six-sided dice. Your strategy is to compute the average wins from 1,000 trials, but the code you've written below produces a `MethodError`.


```

# function to simulate a passadieci roll: 3 6-sided dice
function passadieci()
    # this rand() call samples 3 values from the vector [1, 6]
    roll = rand(1:6, 3)
    return roll
end
# set number of trials and initialize outcome vector
n_trials = 1000
outcomes = zeros(n_trials)
# simulate number of passadieci rolls and count wins
for i = 1:n_trials
    outcomes[i] = (sum(passadieci()) >= 11)
end
win_prob = sum(outcomes) / n_trials # compute average number of wins
@show win_prob;

win_prob = 0.507

```

What does this `MethodError` mean? Why does this code produce one? Fix the error (describe what you changed rather than reproducing the entire code snippet) and solve the problem.

Logical flaw / issue: There are two issues in this code. "at least an 11" includes 11, so "sum(passadieci()) > 11" should be "sum(passadieci()) >= 11" to achieve probability with exactly 11 as well. Additionally, "zero" in julia only returns the integer value 0, which is the source of the `MethodError`. "outcomes" must be initialized as an array or vector for the indexed assignment `outcomes[i] = ...` inside the for loop to work. Otherwise, the loop attempts to assign a value to an indexed position in an integer, producing a `MethodError`. The solution is to change it to "zeros(n_trials)" to return a set vector, and the code above produces the solution ~0.5

Problem 3.3 (3)

You're interested in writing some code to remove the mean of a vector from all of its components. You've written the following code and tried to test it on a random vector, but your code returns a `MethodError`.

```

# function to remove mean from a vector
function remove_mean(vect)
    # function to compute the mean
    function compute_mean(vect)
        element_sum = 0 # initialize sum
        # compute mean and return
        for v in vect
            element_sum += v
        end
        return element_sum / length(vect)
    end
    m = compute_mean(vect) # compute mean
    # return demeaned vector
    return vect .- m
end

```

end

```
random_vect = rand(1_000)  
@show remove_mean(random_vect)
```

```
remove_mean(random_vect) = [-0.4750664987494184, 0.3062816921070738,  
0.08515283003000862, 0.3109008138798066, -0.4289919275259876, -  
0.249130147408967, -0.06855867529345605, -0.04037863169466838, -  
0.13497452616683447, -0.03747781901678138, -0.05504188700817392, -  
0.36720078379566523, 0.3039145331399594, 0.50135621369716, -  
0.31982472867485423, -0.013385568790509672, 0.12847945778241399, -  
0.2278798987043842, 0.41862189766961444, -0.15124280698849712,  
0.14923105457500796, 0.17535150198466054, -0.05425668814062845, -  
0.17886499967112257, 0.03905227048499249, 0.22080163802358654,  
0.35682280994854987, -0.4540746390714877, 0.11900422067120942, -  
0.06545764949382504, 0.4042818483375745, -0.3543424581121052,  
0.2720450877362739, 0.2709874869268504, -0.050765296983823704, -  
0.3488958108175809, 0.2510845561805607, -0.15764986633830924,  
0.43284996170417034, -0.3630778824934998, 0.4645239336870748, -  
0.22865921791766197, -0.35975966991844355, -0.31643780710834535, -  
0.47033226622535684, 0.28479094658178794, 0.5028683517062804, -  
0.2612378846540261, 0.25710383094837475, -0.3132183462497965,  
0.17976987389503984, 0.20865964890439237, 0.3697534392138222, -  
0.19307864466421953, 0.35223600048850645, -0.3219085035798046, -  
0.45316753899182216, 0.058789305565035144, 0.07294726414480779,  
0.4113753916455023, 0.15461379545808285, 0.010805834995543995, -  
0.21014874793633997, 0.35373057376494166, 0.054701388485030256,  
0.012957979368660832, 0.22152853180360954, 0.25724645714438044, -  
0.27212379327417746, 0.11456012403139704, -0.30709069060163274,  
0.4291576579899592, -0.2774279873165336, 0.40949499459837435,  
0.22933389629087186, -0.03422909088193449, -0.04493038213449141, -  
0.3273309735172891, -0.202340888869448, 0.09164933097949302, -  
0.18981600998110448, 0.43894100922936075, -0.08240843291768862,  
0.01559034467959275, 0.20206202946010987, -0.29074862934447365, -  
0.008104555686580084, -0.029272338083652594, -0.4750734931800452, -  
0.2808093947355308, 0.4039321887921973, -0.4249897393598997,  
0.26843325266644236, -0.04250211531297343, 0.08543199362792153, -  
0.2448665654380734, 0.3261681175213069, -0.23433359710955648, -  
0.34864163457175457, 0.15925572736807714, -0.3605658814312003, -  
0.2633832350608585, 0.21782967030247813, 0.007931070826910136, -  
0.028044459730428573, -0.10102743743293474, -0.1472923075111573,  
0.3038409378635115, 0.05847680679924783, -0.09073634340981807, -  
0.17532763938594131, 0.021828937508778012, 0.38559049035035664,  
0.43582884017517576, 0.07404947329897271, -0.26479979535803155, -  
0.40611293267525284, -0.2910596786090548, -0.06305744809878644,  
0.1743946664204944, -0.4584085610058205, -0.17592278602643918,  
0.035650259198729994, 0.11623590449187471, 0.5111841382230455,  
0.4930456941729162, 0.017338266482614495, -0.29944899620718646,  
0.19690511804546607, 0.10074061853965532, -0.3899509970440289,  
0.17483081224695185, 0.4141951850505784, -0.34332976157960415,
```

0.3407585841220607, 0.2209421743737392, -0.3135380951003036, -
0.35575387712090145, -0.3085836669773292, 0.3646680196663562,
0.40085392383406926, 0.35805427031768333, -0.3254026157891021,
0.3017098316053677, -0.026264449728575, -0.19138725759014363, -
0.41917266839832235, -0.1627549983223675, -0.25358563509638, -
0.2958271592092363, -0.1466482325632381, -0.3755294397764938, -
0.032896752976868227, 0.012369423209197317, 0.0891407944106275, -
0.38703101319350064, -0.37980832822730726, -0.40578379086424854,
0.441487036620581, 0.218420732427138, 0.16642107430736475,
0.45301269133312216, 0.12636078351168112, 0.2938085173644618,
0.4671458420220703, 0.5126191148784486, 0.08831836874719667,
0.17165611834546823, -0.21311833297405336, -0.4018670873682668,
0.1568728438114857, 0.4768238294617506, 0.4840571352012625,
0.4250658361994747, 0.1649692581740897, -0.08090582430131932, -
0.3577809251365627, 0.022447336742989488, 0.47293127951736136,
0.0005738785262036084, -0.04692587995749975, -0.050652808233987434,
0.2969810793695161, 0.025577625066488863, -0.38111737392410294,
0.17333295050511666, 0.4957097885525661, -0.28174773636143013,
0.21395753870098622, 0.078008283337892, 0.397926853927429, -
0.358766053709197, -0.01282246309498336, 0.004157637428028216, -
0.05066738914424784, 0.32291471656653015, -0.0025609353265676482,
0.22431997551043492, -0.09467549581272938, 0.3654946992021687, -
0.13825117832162392, 0.16668369297434638, 0.4499624335065977,
0.35296414672459975, -0.48065142553513074, -0.2267486568651469, -
0.10277894986821645, -0.2886402910349648, 0.45930113325227384,
0.14825368226824143, -0.3183028017536945, -0.3268693868596648,
0.15701776729390393, 0.11863908955506475, 0.30322988698534503, -
0.1322378360457982, -0.3039886569101111, 0.09959171618665286,
0.15090790963160905, 0.3144157902398633, -0.16320035856671566, -
0.38728229396422575, 0.07162232352296694, 0.45950788548945964, -
0.4180401004867019, -0.017766790256626308, 0.3037512715169086, -
0.4213604422424455, 0.16387605718351927, -0.12514446729924378, -
0.060354968085494676, -0.3803143768092412, 0.22486846451319165, -
0.017962999196978524, -0.09526827745166555, 0.3176495197696737, -
0.4134866630218703, -0.35830329701575614, -0.04701083582737242,
0.012296947986329299, -0.0010327058305741321, 0.4291731392581418,
0.18100335887266983, -0.3909861146627409, -0.3500817504604453, -
0.05206729173826641, -0.08417623789710171, -0.3309587113982241, -
0.33143448065401226, -0.28026459198048803, -0.31231379659293423, -
0.4054588478305926, 0.3393168735362163, 0.5144704951847948, -
0.40067705875080395, 0.23242253133548452, 0.33442300024459815,
0.2590996771996902, 0.5036476947442545, -0.017017244944766086,
0.02603856950263328, 0.16300824474394338, -0.19292165331853262, -
0.15211694075289717, 0.4724121533050384, 0.2756674320928939,
0.48487584795112804, -0.33335593323065715, -0.30056818154214615, -
0.3393931022566604, 0.41132261105901524, -0.43878279540686893, -
0.0348128057493019, -0.4196660454092149, -0.12495832589288736,
0.25421988350754476, -0.2229689559582152, 0.04105728178132939,
0.34051169409227156, 0.41337076100563397, 0.4073074495147392,

0.3090751383858181, -0.28406519077861836, 0.13392634654610946,
0.4889772878765989, -0.10331159393666589, 0.06894861041766387, -
0.41394988916051867, 0.4686308318308682, 0.3397133552194699,
0.5083992275187895, 0.21765673105211525, -0.07175327539880383,
0.3161730441743983, 0.45811381991856903, 0.3955384752990887,
0.11875143765558671, 0.13182551296707046, -0.3119228324211335, -
0.34429537849919134, 0.0887956704866979, -0.1068031751779076, -
0.007542179720517184, -0.2513757623764067, 0.04807535404901242,
0.3902254887827036, 0.3869051455383913, -0.16681508728800065, -
0.10678635685598481, -0.0992330289881459, -0.009984968936996519, -
0.39982172944282757, -0.33707917746592, -0.15954507114124394, -
0.20044568874841595, 0.2644482801820538, -0.3024670589424452,
0.20939322725326237, -0.08863196637478965, -0.3393740206477146,
0.45858958843482533, 0.3903221167876417, -0.19960682492420345, -
0.2619329526596975, -0.17752005454665631, 0.026870782017872852,
0.3361727934927793, -0.15049615143614103, -0.23476030241210444,
0.4307057255500253, 0.26454340956407796, -0.458512968621932, -
0.06801203134634792, 0.0321992367186853, 0.01371921264574516, -
0.051169025053788064, -0.2891651206292879, 0.21746054487375222,
0.13701428853653608, 0.22684696948207306, 0.10425715613638198, -
0.07140720449266313, -0.016194275646732037, 0.3126120182092401,
0.4752287373106372, 0.2699968681911188, 0.28492218719253615, -
0.05908353245279163, 0.1754976055533789, -0.329258032127013, -
0.19365977356574782, -0.30418696150100055, -0.40337150320508586, -
0.1391332958662872, -0.18889463733598055, 0.3972957429014303,
0.24675712651883308, 0.09742521788258085, -0.13460273003867185,
0.21358914033253518, 0.20082303937218082, 0.00780747688420802,
0.04279946047323946, -0.15479430808088002, -0.39551201651616785, -
0.3880052767783143, 0.019234620774844058, -0.02321632511471894, -
0.2555824529458709, -0.3585484329455427, -0.18843370728780195,
0.4259644061629696, 0.09319342489520455, -0.11826239019960005, -
0.11320870637460212, -0.24191896493401843, 0.08963639085767416,
0.4929064097345527, -0.11124716186229122, 0.4384260144058012,
0.17212039708339233, 0.4728801308921199, 0.08656672526051135,
0.22359519224168378, 0.46174752631763616, -0.2839347546227825,
0.2157702734803526, 0.05201027780707557, -0.3847085257382935,
0.27892725043690125, 0.5007168167169104, 0.30289036002074166, -
0.020065058653723877, -0.03738016959617152, 0.2011428764589378, -
0.4580401648785898, -0.09343940920646088, -0.16663934711390527, -
0.2532620486672882, -0.35855487580018963, 0.25043299993363277, -
0.011892064710820827, 0.2835786461612124, 0.2663907663666061, -
0.023287181380927446, 0.4617513597243378, 0.4968084531953778, -
0.08640564111431015, 0.07120165189392247, -0.28903995088496504, -
0.39530030155966833, 0.07727134857617257, 0.052068285818086646, -
0.24866259852804284, 0.3039517020841679, 0.1498305061782067,
0.1324538016098401, -0.24999167337042083, 0.18966939017366224, -
0.2703752747490926, -0.22675453248879185, 0.11680453592274931, -
0.24703283480888938, -0.46110955400945575, -0.2679645626901359,
0.3158557318157873, -0.4653738312048713, -0.0580216910891842, -

0.36734042439668024, 0.02523695109096824, 0.1934538090761443,
0.30042694712583373, -0.14895814854989287, -0.06638527141828132,
0.26210221873423933, 0.4550675327965864, -0.3638145103624406,
0.27026681410261355, -0.4388785284038894, -0.1290528993377532,
0.0008906455104683575, -0.082236463715599, -0.22302377837561316, -
0.24440917297675657, -0.4744630104336832, -0.11470908336917518,
0.5098813277942812, -0.4134429693192332, -0.2790080945871368,
0.07316075211979123, 0.012620664766880285, -0.08005395292775674, -
0.4779606559697105, 0.47646831369315457, 0.2950595047619051, -
0.070425229749981, -0.11742991437012462, -0.3716367442325408, -
0.31084434320207643, 0.18553200749080412, 0.32937027379889916, -
0.3427594121899469, -0.12516837456704244, -0.2637072340866393,
0.13195099302608704, 0.16933009638822083, -0.05210572666969737, -
0.4297003332471604, 0.3096634746517246, 0.03583228007651573,
0.3943801696122353, 0.43662621491898557, 0.04397350554687285, -
0.42819041692226134, -0.39229467218483804, -0.06650187903381699,
0.39595147363093175, 0.10987617512902514, 0.35797990892856313,
0.4395328262770779, 0.4981883977027388, -0.32373885085088683, -
0.3449802755699578, 0.22335661718620003, 0.2528573485877811, -
0.16261444220418142, -0.18381916856205038, 0.11635048757985689,
0.4933443412928805, -0.19475948876864224, -0.36581236515077764,
0.4777630943662319, -0.41537028809777177, -0.2641364925398626, -
0.028736167100999133, -0.4687052101028466, 0.13197171391403517, -
0.403968322657246, -0.25143380627558665, 0.4978934639440966,
0.000605513291264792, -0.3111133140785367, 0.32049923114920453,
0.4480763631618466, 0.20629591073961684, -0.2765386241716037,
0.09498632046613231, -0.36246986431381023, 0.43616026299919197, -
0.413053527285021, 0.39582134382622736, 0.22372992387706114,
0.0680612054748101, 0.03305752570801446, 0.16334846831109728,
0.01215388984302912, 0.06068553530275378, -0.3866717020235686,
0.1669333784507825, -0.04000126372020618, -0.06304807743861884,
0.3802454718764998, -0.3864310384164984, 0.3202947703478777,
0.03883768017358302, 0.0018659962849835177, 0.03973211454144299,
0.30165222690069793, 0.012330890368705716, -0.10401836902608885,
0.5124606408701735, -0.4621858667586326, -0.16143818703941648, -
0.429079038314755, -0.005688083535788202, -0.17392564532491894,
0.27056695416590304, 0.4704727019013272, -0.1879732473138983, -
0.06328005146514293, -0.06579865704774679, -0.2389581608287803,
0.17498026487873408, 0.48513335086607057, -0.07154438382720285,
0.367415151675206, 0.37104245201751473, 0.487772347420821,
0.06272867738527066, -0.13148636393433277, -0.13322472323999124, -
0.29566955530048267, 0.040996755915066685, -0.34328283827067685, -
0.3272810401600693, 0.052390114912189034, -0.12079227822455196, -
0.40900496379371265, 0.4883907786364773, -0.3920808776465524,
0.025195702498597816, -0.09733225396951717, 0.37838878730458914,
0.08587798115191181, 0.5018516003332756, -0.38847691496603876,
0.514506086409718, 0.24674475705104215, 0.32805530665917304, -
0.1290257339751747, 0.26108712504372966, -0.1266925080112128, -
0.11609463247945728, 0.20739664226826038, -0.38854506384568843,

0.2190574043043816, 0.22736345815866188, -0.2467877894257966, -
0.0529853189620374, 0.06647158725354718, -0.3006354321106769, -
0.15713287648026675, 0.02685876794704045, 0.45078531400221167,
0.24610087994545193, 0.11469931969131042, -0.08415033109894376,
0.363695908873374, -0.0601895858072351, -0.44890047494987484, -
0.41077011561046806, -0.09468218013611784, -0.34425537718834653, -
0.13277962757895634, -0.471075969980177, -0.010829412851406706,
0.16221079032275987, -0.35211464560911965, -0.011047157354525328, -
0.11425535626153993, -0.2821668620938006, 0.4241682990665716, -
0.24307147401908424, 0.30834327376135495, 0.1197143755207799, -
0.1820598770745584, -0.1548214423186286, 0.1745891105502615, -
0.46957712333561885, -0.29917588642270276, 0.08763149206142551,
0.3287034500304643, 0.1192318882466753, -0.2982369528874135,
0.4333568992051945, 0.3800276068703682, -0.21598346805658097, -
0.16034929792743113, 0.12255539871376397, 0.09291220392713717, -
0.4520520751938102, -0.4363227413774815, 0.4116612445132759, -
0.43740004896987394, 0.22169279601088354, 0.10689236986063744, -
0.2794207808328224, 0.014534431443942175, -0.15427008723161184,
0.1275754896424175, 0.20054881106494316, 0.019966725534920182, -
0.400785640085082, -0.21065287166817304, 0.35057899178482543, -
0.21074891593816314, 0.11871345068867367, -0.3484793684340236, -
0.4007955868387729, 0.49673123092346383, -0.31592510508010185,
0.3580493213163374, -0.30985797982001406, -0.3695490314627484,
0.08837015809147519, 0.31842677150264, -0.03269090898811289,
0.08134507705885374, -0.3070543654503788, -0.2608638940434155,
0.3604709296565203, 0.42542575881794853, 0.3673770174902007,
0.35264010953536007, -0.02151956995558657, -0.05991513476408328, -
0.39655142637823126, -0.07145044543756396, 0.1312955599227008,
0.40919510574943996, -0.20415434748688066, -0.17259314632706502, -
0.11987613194668989, -0.33148975951569537, -0.24044003528975877, -
0.23991016919118913, 0.24981105378249846, 0.3307802817149943,
0.1190691672615104, 0.4748479858152066, 0.23055949515377305,
0.38320994718255685, -0.4007642202586268, -0.2979892591368134, -
0.09115107482080076, -0.32877443397312156, 0.35208890997670217,
0.4162085755891759, 0.0049802778831546934, -0.4124926391428504,
0.47153059502557626, 0.5095314514398186, -0.2954260353712913, -
0.289803691017651, 0.4299791648092306, 0.2571660108776227, -
0.4748638071441714, -0.18670119447618494, 0.18518408631821415,
0.14003957143793733, 0.1136641329845296, -0.33110530996819765, -
0.09613983032104245, 0.4621335103658266, 0.28325992538926814,
0.23119464644539334, -0.32045461455872337, 0.46302685519213505,
0.3932836265866483, 0.17572380059324855, 0.10939070539773688, -
0.3844805003981995, -0.34503260095660904, -0.43888977154401076, -
0.34064867048507186, 0.46822913429672763, -0.44398285914250657, -
0.1867682810196093, -0.12152092632590605, 0.3082927433336441, -
0.24984676974886721, -0.1957320955258609, -0.3515957818264889, -
0.0639320655437986, 0.09603491688351795, 0.12214304096487694,
0.45316833983639804, -0.13363770570059885, 0.11575708683995578,
0.19606236244365816, -0.09297755114515993, -0.36877348963389345,

0.48621156782468056, 0.2426065792426817, 0.25828518097640385,
0.47837190187771894, -0.2393027657969159, 0.3242596889199234,
0.33500706341681386, -0.3660753325298849, -0.14216584790775932, -
0.428823218286566, 0.3165736162791336, -0.42492446296890973,
0.30560384088451287, -0.4313370344756242, 0.39239269664480714, -
0.20443973806889804, -0.20938525770268335, 0.18266532769162458,
0.39804496581565973, 0.446680393580705, -0.32383098591721926,
0.254894642710632, 0.4012506810266224, -0.4644298477401786, -
0.3858577325577097, 0.08208282725225935, 0.2236947436128387,
0.10776464192776103, -0.19798291010276847, -0.38606724418203275, -
0.4593110167515083, 0.19069220333527204, 0.1603423633347556,
0.2842557084620778, 0.496284492037369, -0.2775166176476195,
0.285859226309359, -0.0905317668404827, 0.3095204626010696, -
0.291441245784528, 0.03331820188450285, -0.16493541209060425, -
0.2314146186506455, -0.4267501255865006, 0.10184544849777255, -
0.06977374416012472, -0.014959294858559846, -0.10341114540737795,
0.08717906581207358, -0.4680707369545386, 0.4760328559251329, -
0.01771518468504063, -0.010789231672209132, 0.3289502967582708, -
0.0906938065896401, 0.36442645052017797, 0.3736356400287455, -
0.3087025181348496, 0.048392069585606046, 0.15589880600802763,
0.11127846702614697, 0.31234419910449196, -0.16885264875489303, -
0.17970235923993527, -0.33036824471324466, -0.3480500712543113,
0.1883480273544147, -0.099667812177334, -0.26417563842442515, -
0.3097467834068033, 0.16740865404418503, -0.10516786061240957,
0.5112614715017412, -0.028336998958549797, 0.07566081068485675,
0.09750090670388634, -0.20696660040281578, 0.4366452966068207,
0.309362014921843, -0.015411082823743705, -0.4045617303403314, -
0.28958577284302633, -0.07479845063015422, -0.1374602869761984, -
0.47825800760429793, 0.08198874092720554, -0.18940095806934998, -
0.22924351437717239, 0.09428812590731539, -0.42437194348164897,
0.2603502134906672, -0.45924936071284894, -0.26592259895123704, -
0.023534462879343687, 0.2812583597003314, 0.11047616383223291, -
0.42172103304336905, 0.24335988424128674, -0.056678129335693384,
0.09399072422831578, -0.26844684909196403, -0.2870003413437495, -
0.33338739979180443, -0.12258875210349662, -0.45231254642374397, -
0.4091478084457978, -0.24030404299980757, 0.3262022148762187, -
0.20295652642852835, -0.37116489808840225, -0.3026776313973816, -
0.18222888842555862, -0.07561120912373831, -0.2699120702496365,
0.13194248750427784, 0.16794585692968267, 0.01911285246537653,
0.1804758782454946, -0.19509926098995178, -0.23083650287077995, -
0.24551027281363513, 0.12288988503452025, -0.18256455656465498, -
0.39313429182677995, 0.4891235997239385, 0.5093939156633277, -
0.2666759972054914, -0.43844637472710035, 0.5184886498491508, -
0.10903062063139213, 0.2321310674086149, -0.2998673855222691, -
0.23247791391878536, -0.15522007025209406, 0.43975848890140745, -
0.4199575821035666, 0.011159755507428093, 0.15677730100653353,
0.21803708741521843, 0.06099147802050309, -0.47701519336198595,
0.5148418985205488, -0.21131661143042946, -0.23985798918547196,
0.30945226126365055, 0.2840632386562011, 0.344021799110069,

0.3606459724155837, 0.29626100067264705, 0.48988272564561486,
0.43850668278282534, -0.169487516299478, -0.2912001820951958, -
0.21005598514820678, 0.46101118795962037, 0.2300720509587102,
0.21652280724997597, 0.5040484177537521, -0.19163965100260127, -
0.1704166244321953, -0.13998277309115315, 0.019271716235068737, -
0.46577414172286713, 0.23805620072052192, 0.13601056909161502, -
0.2832861359935127, 0.4143056695992181, 0.41406448137759366, -
0.07132630107670768, -0.28893284435021827, -0.17462541968531076,
0.4658139488407126, -0.08139008219100052, -0.26220047084694553, -
0.468767656554964, 0.3367382211861166, -0.15907250111550297, -
0.302798562928252, 0.14827935207726883, 0.004042771758667518, -
0.3001348545052621, 0.19828855505798726, -0.33617202833304505, -
0.3744115358300738, 0.027122366979726475, -0.041936682199886666, -
0.08315992006261463, 0.46289018781416247, 0.3091945472176857,
0.33472541293682156, -0.4769783559124311, -0.3676071251971551,
0.4385849151489345, -0.2399599374810234, 0.2640724983742654,
0.08660290376897606, -0.25279316523608264, 0.052977803465310336, -
0.4667349566391047, -0.38795148280240255, 0.4952761510263358,
0.5179507831660731, -0.1890493966265342, -0.05130461755619753,
0.052446126215857425, -0.277686699886295, -0.3415706549644303,
0.4631326280843556, 0.2648223628130585, -0.4209112678799123, -
0.08360718508367637, 0.11608474352787057, 0.043921341790492896,
0.2270391545366578, -0.2927567688066949, 0.3808323892055804,
0.4603817641078026, -0.07228530290543711, 0.08936619480522001, -
0.4589159213219065, 0.26407178526033037, -0.1270500135409306, -
0.3426070101844306, -0.16609810559393723, 0.05569517318072581,
0.4954923353884253, -0.2344981126157093, 0.42391023259496974, -
0.25438614042819885, -0.03543068451521425, -0.02782931136151523, -
0.3615762542864823, 0.009609859749809213, -0.09768886795618814, -
0.3898504163716836, -0.3989278213267897, -0.24026837361261733, -
0.4270679246163611, -0.47922097606925707, -0.45794523610418314,
0.09687582283386509, 0.06169363259319127, -0.03899695916895307, -
0.39090528790981716, -0.3125909827850303, -0.3258132891230606,
0.3212295236600029, 0.2130556142090425, -0.29896459067608094, -
0.2083288621005306, 0.21006967917756947, -0.29899455390395047,
0.37394963443633633, 0.394613118759516, 0.3677980823745062, -
0.4691927964011223, -0.4196165319673175, -0.3954559542999878,
0.5066484455789929, -0.35253641308523054, 0.009765157675862834, -
0.4004483969461127, -0.06572576773288186, -0.041116511444503234, -
0.055081733018509604, 0.036376148974782696, -0.4263861895444695,
0.42714856098416476, -0.14220960329609256, 0.04726602597649732,
0.022466452771657708, 0.3587784126895489, 0.4736680301449798, -
0.23949598991395105, -0.46047579925291815, 0.49879736520282425, -
0.17928148087113616, 0.47812228465539364]

1000-element Vector{Float64}:

-0.4750664987494184
0.3062816921070738
0.08515283003000862
0.3109008138798066


```

-0.4289919275259876
-0.249130147408967
-0.06855867529345605
-0.04037863169466838
-0.13497452616683447
-0.03747781901678138
-0.05504188700817392
-0.36720078379566523
 0.3039145331399594
  :
-0.4263861895444695
 0.42714856098416476
-0.14220960329609256
 0.04726602597649732
 0.022466452771657708
 0.3587784126895489
 0.4736680301449798
-0.23949598991395105
-0.46047579925291815
 0.49879736520282425
-0.17928148087113616
 0.47812228465539364

```

What does this `MethodError` mean? Why does this code produce one? Fix the error (describe what you changed rather than reproducing the entire code snippet) and solve the problem.

Logical flaw / issue: The line `"return vect - m"` is attempting to subtract a single number (`m`) from a vector (`vect`), which is not possible to directly compare. This is the `MethodError` issue in the code. As a solution, Julia has the capabilities to remove `"m"` from every element within the vector with proper syntax. The code above uses the proper Julia syntax `(.-)`, and produces a value close to 0 within every element of the vector. This makes logical sense if you want to remove the mean of the vector from each element.

Problem 4 (5)

Loading and Manipulating Data

The file `fha.csv` contains data[1] on the size of American metro areas and the number of miles people drive in each metro area each day. This problem asks you to load and work with this data to get practice with data manipulation in Julia.

Problem 4.1 (2)

Load `fha.csv` into a Julia `DataFrame`. How many rows and columns does the data have?

Solution: 492 rows and 5 columns!

Problem 4.2 (3)

Calculate the number of miles driven per person per day for every metro area. Make a scatterplot of this on the `y`-axis and the population of the city on the `x`-axis. Make sure to label

your axes and include a caption clearly describing the plot. Highlight where the New York City and Houston metro areas are on the plot and make a legend clearly labelling any plot features.

#Question 4.1

```
fha = CSV.read("data/fha.csv", DataFrame)
```

```
@show size(fha)
```

```
size(fha) = (492, 5)
```

```
(492, 5)
```

#Question 4.2

```
fha.miles_per_person_day =  
    fha[:, "daily_vmt (mi/day)"] ./ fha.population
```

```
p = scatter(  
    fha.population,  
    fha.miles_per_person_day,  
    label = "Metro areas",  
    color = :gray,  
    alpha = 0.5,  
    markersize = 3,  
    xlabel = "Metro-area population",  
    ylabel = "Daily vehicle miles traveled per person  
(mi/person/day)",  
    title = "Daily Driving per Person vs. Metro-Area Population"  
)
```

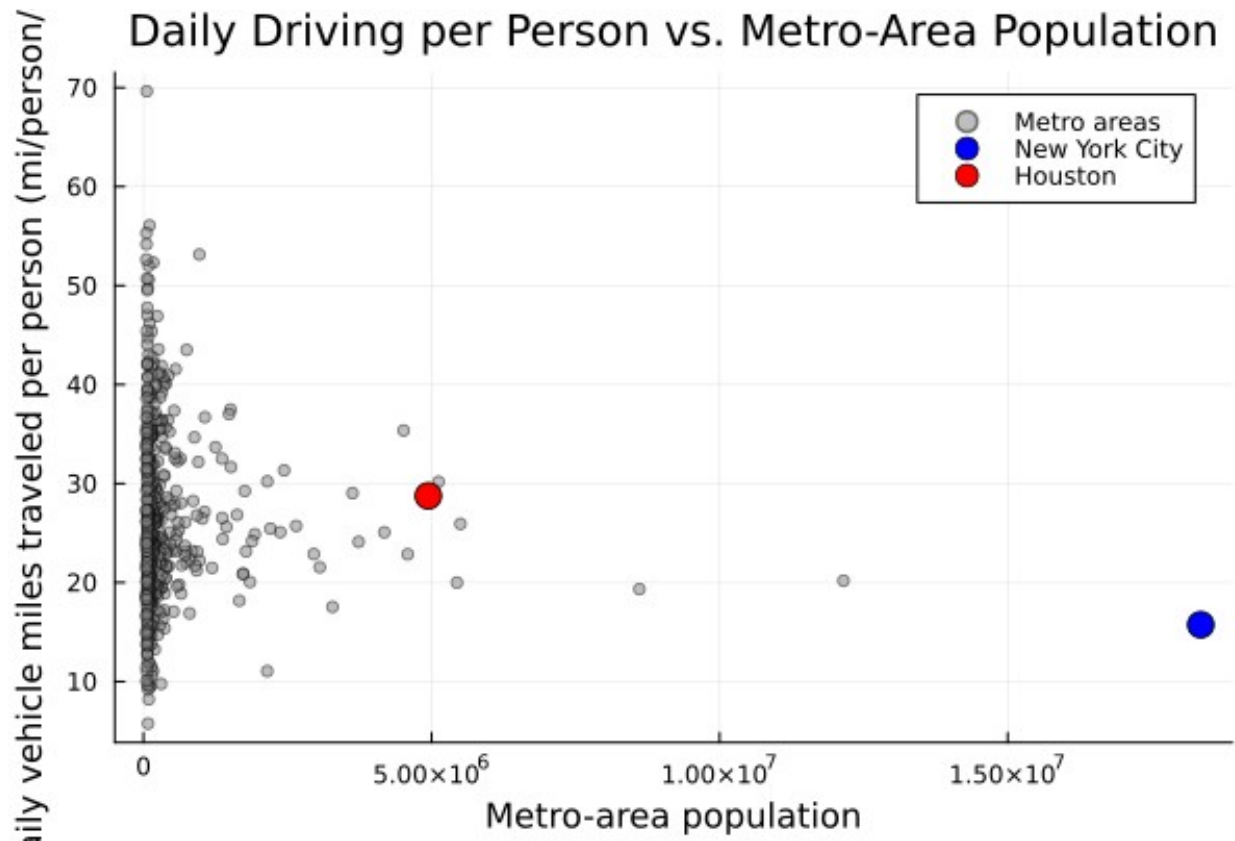
```
nyc = occursin("New York", fha.metro_area)  
houston = occursin("Houston", fha.metro_area)
```

```
scatter!(  
    p,  
    fha.population[nyc],  
    fha.miles_per_person_day[nyc],  
    label = "New York City",  
    color = :blue,  
    markersize = 7  
)
```

```
scatter!(  
    p,  
    fha.population[houston],  
    fha.miles_per_person_day[houston],  
    label = "Houston",  
    color = :red,  
    markersize = 7  
)
```

)

```
display(p)
```



Problem 5 (20)

Computational Modeling.

Cheap Plastic Products, Inc. is operating a plant that produces $100 \text{ m}^3/\text{day}$ of wastewater that is discharged into Pristine Brook. The wastewater contains 1 kg/m^3 of YUK, a toxic substance. The US Environmental Protection Agency has imposed an effluent standard on the plant prohibiting discharge of more than 20 kg/day of YUK into Pristine Brook.

Cheap Plastic Products has analyzed two methods for reducing its discharges of YUK. Method 1 is land disposal, which costs $X_1^2/20$ dollars per day, where X_1 is the amount of wastewater disposed of on the land (m^3/day). With this method, 20% of the YUK applied to the land will eventually drain into the stream (*i.e.*, 80% of the YUK is removed by the soil).

Method 2 is a chemical treatment procedure which costs $\$1.50$ per m^3 of wastewater treated. The chemical treatment has an efficiency of $e = 1 - 0.005 X_2$, where X_2 is the quantity of wastewater (m^3/day) treated. For example, if $X_2 = 50 \text{ m}^3/\text{day}$, then $e = 1 - 0.005(50) = 0.75$, so that 75% of the YUK is removed.

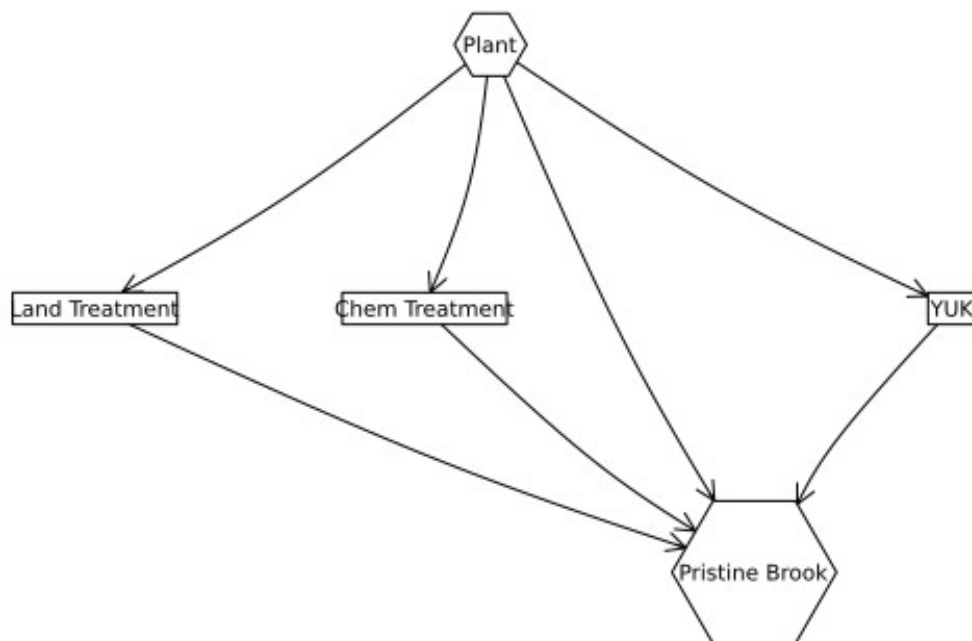
Cheap Plastic Products is wondering how to allocate their wastewater between these three disposal and treatment methods (land disposal, chemical treatment, and direct disposal) to meet the effluent standard while keeping costs manageable. The flow of wastewater through this treatment system is shown in Figure 1.

[1] From the [2023 Federal Highway Administration's Highway Statistics report](#)

```
A = [0 1 1 1 1;
      0 0 0 1 0;
      0 0 0 1 0;
      0 0 0 0 0;
      0 0 0 1 0]

names = ["Plant", "Land Treatment", "Chem Treatment", "Pristine
Brook", "YUK"]
# modify this dictionary to add labels
edge_labels = Dict{(1, 2) => "", (1,3) => "", (1, 4) => "", (2, 4) =>
"", (3, 4) => "", (1, 5) => "", (5, 4) => ""}
shapes=[:hexagon, :rect, :rect, :hexagon, :rect]
xpos = [0, -1.5, -0.25, 1, 1.75]
ypos = [1, 0, 0, -1, 0]

p = graphplot(A, names=names, edgelabel=edge_labels, markersize=0.15,
markershapes=shapes, markercolor=:white, x=xpos, y=ypos)
display(p)
```



Problem 5.1 (2)

Make a figure showing notation for how YUK moves through the system for a given treatment allocation. Make sure to define your variables. You can make this by modifying the graph above or with an independent figure included in your solution.

You can modify the edge labels (by editing the `edge_labels` dictionary in the code) in Figure 1 to show how the wastewater allocations result in the final YUK discharge into Pristine Brook. For the `edge_label` dictionary, the tuple (i, j) corresponds to the arrow going from node i to node j . The syntax for any entry is $(i, j) \Rightarrow \text{"label text"}$, and the label text can include mathematical notation if the string is prefaced with an `L`, as in `L"x_1"` will produce x_1 .

Problem 5.2 (4)

Formulate a mathematical model for the treatment cost and the amount of YUK that will be discharged into Pristine Brook based on the wastewater allocations. This is best done with some equations and supporting text explaining the derivation. Make sure you include, as additional equations in the model, any needed constraints on relevant values.

None

You can find some basics on writing mathematical equations using the LaTeX typesetting syntax [here](#), and a cheatsheet with LaTeX commands can be found on the course website's [Resources page](#).

Let X_1 represent wastewater sent to land disposal, X_2 represent wastewater sent to chemical treatment, and X_3 represent wastewater discharged directly to Pristine Brook, all in m^3/day .

The total treatment cost is:

$$C = \frac{X_1^2}{20} + 1.50 X_2.$$

The chemical-treatment efficiency is:

$$e = 1 - 0.005 X_2.$$

Therefore, the total YUK discharge is:

$$Y = 0.2 X_1 + 0.005 X_2^2 + X_3.$$

Complete Mathematical Model:

$$\begin{array}{ll} \min_{X_1, X_2, X_3} & \frac{X_1^2}{20} + 1.50 X_2 \\ \text{subject to} & X_1 + X_2 + X_3 = 100, \\ & X_1, X_2, X_3 \geq 0. \end{array}$$

Problem 5.3 (3)

Implement your systems model as a Julia function which computes the resulting YUK discharge and cost for a particular treatment plan. You can return multiple values from a function with a [tuple](#), as in:

```
function multiple_return_values(x, y)
    return (x+y, x*y)
end

a, b = multiple_return_values(2, 5)
@show a;
@show b;

a = 7
b = 10
```

To evaluate the function over vectors of inputs, you can *broadcast* the function by adding a decimal `.` before the function arguments and accessing the resulting values by writing a *comprehension* to loop over the individual outputs in the vector:

```
x = [1, 2, 3, 4, 5]
y = [6, 7, 8, 9, 10]

output = multiple_return_values.(x, y)
a = [out[1] for out in output]
b = [out[2] for out in output]
@show a;
@show b;

a = [7, 9, 11, 13, 15]
b = [6, 14, 24, 36, 50]

# Compute YUK discharge and cost for one wastewater allocation/run
function treatment_model(x1, x2, x3)
    # YUK reaching the brook from each disposal method
    land_yuk = 0.20 * x1
    chemical_efficiency = 1 - 0.005 * x2
    chemical_yuk = (1 - chemical_efficiency) * x2
    direct_yuk = x3

    # Total YUK discharge in kg/day
    yuk_discharge = land_yuk + chemical_yuk + direct_yuk

    # Total treatment cost in dollars/day
    treatment_cost = x1^2 / 20 + 1.50 * x2

    return (yuk_discharge, treatment_cost)
end
```

```

#For 1000 Runs:
Random.seed!(1000)

# Each column is a run whose components add to 100 m³/day
runs = 100 .* rand(Dirichlet(ones(3)), 1000)

x1 = runs[1, :] #land disposal
x2 = runs[2, :] #chemical treatment
x3 = runs[3, :] #direct discharge

#For combining results to all 1,000 runs
results = treatment_model.(x1, x2, x3)

yuk_discharge = [result[1] for result in results]
treatment_cost = [result[2] for result in results]

# Identify what meets the 20 kg/day standard
compliant = yuk_discharge .<= 20
compliantCosts = treatment_cost[compliant]
#@show results;
#@show compliant;
@show compliantCosts;

p = scatter(
    treatment_cost[.!compliant],
    yuk_discharge[.!compliant],
    label = "Exceeds standard",
    color = :red,
    alpha = 0.6,
    markersize = 3,
    xlabel = "Treatment cost (\$/day)",
    ylabel = "YUK discharge (kg/day)",
    title = "YUK Discharge versus Treatment Cost"
)

scatter!(
    p,
    treatment_cost[compliant],
    yuk_discharge[compliant],
    label = "Meets standard",
    color = :green,
    alpha = 0.7,
    markersize = 3
)

hline!(
    p,
    [20],
    label = "20 kg/day effluent standard",

```

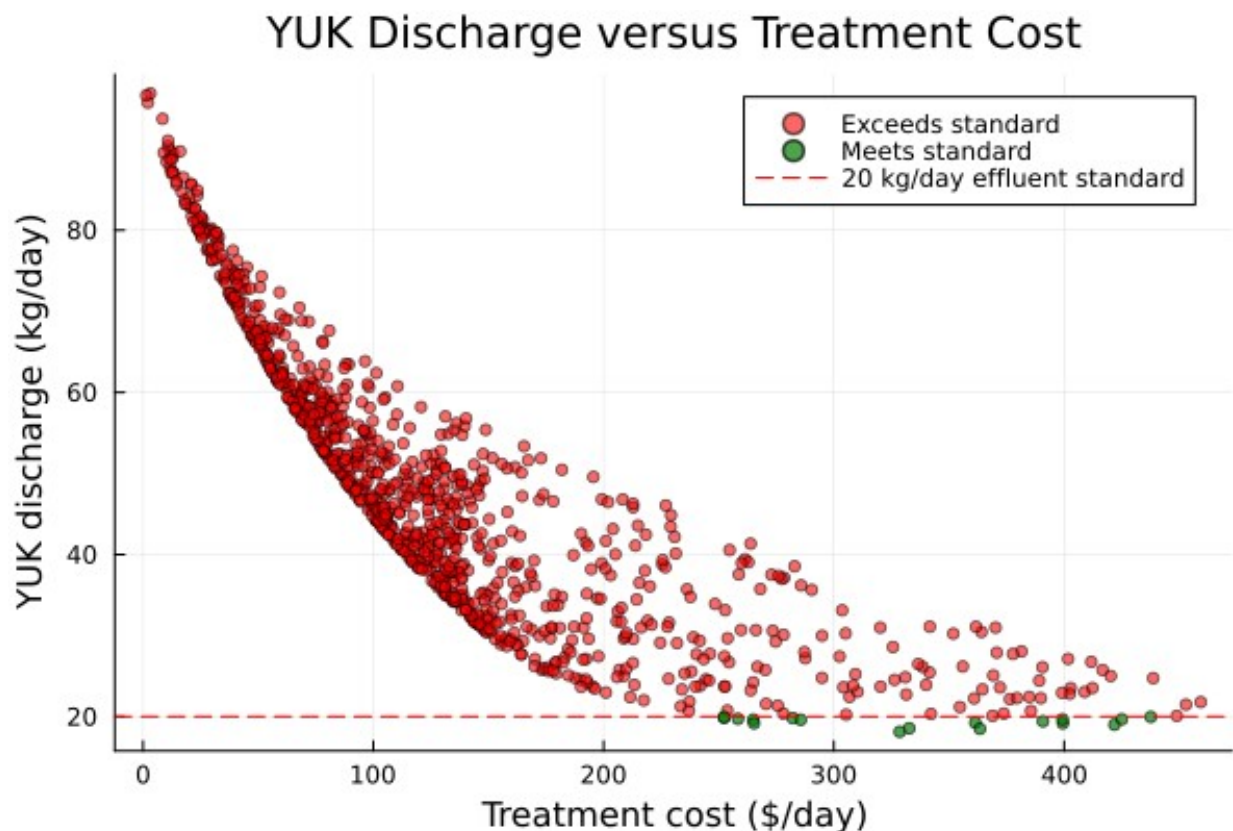
```

        color = :red,
        linestyle = :dash
    )

display(p)
print("YUK discharge into Pristine Brook versus daily treatment cost
for 1,000 randomly generated wastewater runs. Green points satisfy the
20 kg/day effluent standard, while red points exceed it. The dashed
line represents the maximum permitted YUK discharge")

compliantCosts = [361.41892420129324, 399.4387913959372,
399.30782167667957, 363.43568869932005, 265.08640664509534,
421.8756020033286, 390.8233922659977, 437.6525217598731,
265.342957704057, 258.3404537659346, 425.060030935176,
282.22229823070865, 252.3035892080785, 252.3710904790684,
332.7329882060366, 328.5980581327737, 285.62152257750887]

```



YUK discharge into Pristine Brook versus daily treatment cost for 1,000 randomly generated wastewater runs. Green points satisfy the 20 kg/day effluent standard, while red points exceed it. The dashed line represents the maximum permitted YUK discharge

#Question 5.6:


```

# Analytical minimum-cost strategy
min_cost_yuk, min_cost = treatment_model(0, 0, 100)

# Analytical minimum-YUK strategy
min_yuk, min_yuk_cost = treatment_model(80, 20, 0)

@show min_cost, min_cost_yuk
@show min_yuk_cost, min_yuk

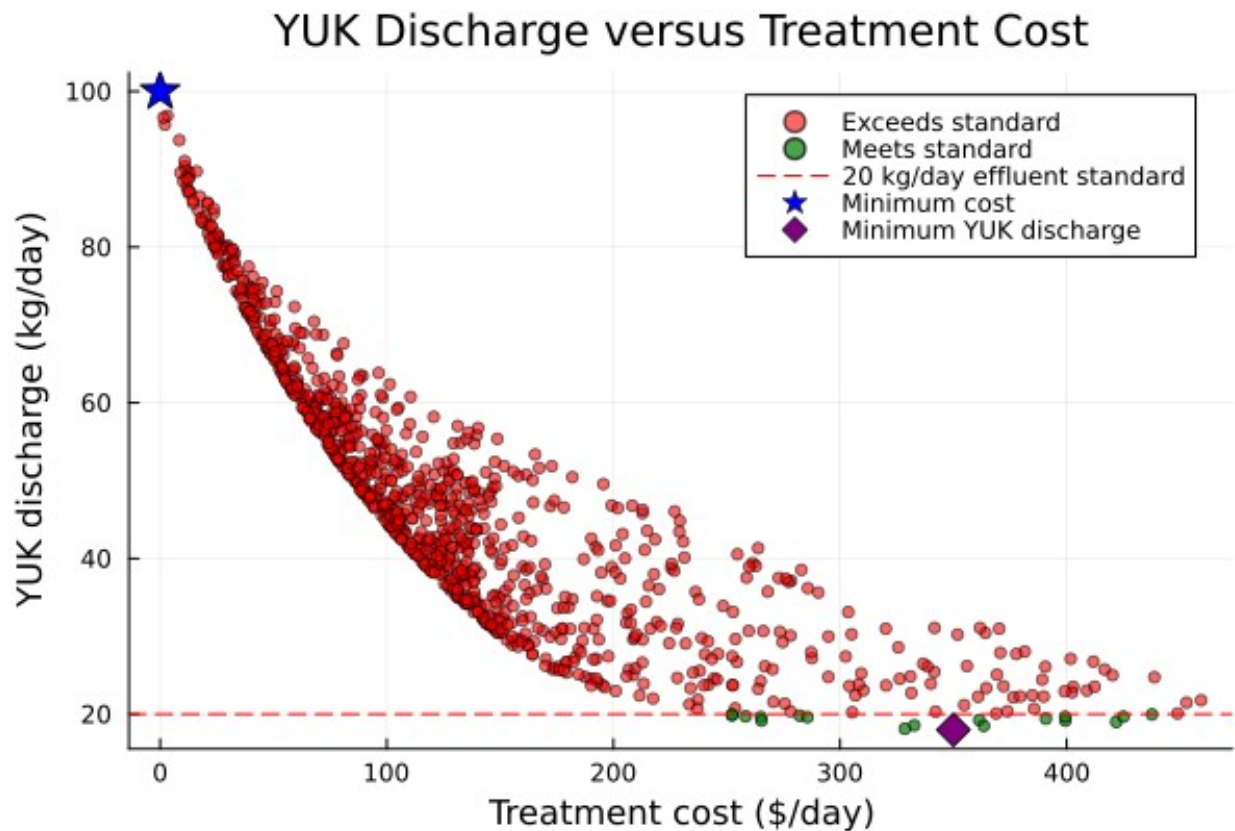
scatter!(
    p,
    [min_cost],
    [min_cost_yuk],
    label = "Minimum cost",
    color = :blue,
    marker = :star5,
    markersize = 10
)

scatter!(
    p,
    [min_yuk_cost],
    [min_yuk],
    label = "Minimum YUK discharge",
    color = :purple,
    marker = :diamond,
    markersize = 8
)

display(p)

(min_cost, min_cost_yuk) = (0.0, 100.0)
(min_yuk_cost, min_yuk) = (350.0, 18.0)

```



Make sure you comment your code appropriately to make it clear what is going on and why.

Use your function to experiment with 1,000 different combinations of wastewater discharge and treatment. You can do this with either a grid search or by sampling from a [Dirichlet distribution](#) (a Dirichlet $(1, n)$ distribution will generate uniformly-weighted n -dimensional vectors whose components add up to 1; see the [Distributions.jl documentation](#) for how to sample from probability distributions in Julia).

Plot the results of these experiments by making a scatterplot of YUK discharge into the lake vs. treatment cost. Make sure to label your axes and include a legend, as well as a caption clearly describing the plot.

Problem 5.4 (3)

Do any satisfy the YUK effluent standard (plot this as well as a dashed red line). What was the cost of solutions satisfying the standard?

Yes! Costs of solutions were between \$252.3 and \$437.65

```
compliantCosts = [361.41892420129324, 399.4387913959372, 399.30782167667957,
363.43568869932005, 265.08640664509534, 421.8756020033286, 390.8233922659977,
437.6525217598731, 265.342957704057, 258.3404537659346, 425.060030935176,
282.22229823070865, 252.3035892080785, 252.3710904790684, 332.7329882060366,
328.5980581327737, 285.62152257750887]
```

Problem 5.5 (4)

What can you say about the tradeoff between treatment cost and YUK discharge? You don't have to find an "optimal" solution to this problem, but what do you think would be needed to find a better solution?

The results show a clear tradeoff between treatment cost and YUK discharge. Less treatment is cheaper but releases more YUK, while greater treatment usually lowers discharge at a higher cost. However, spending more does not always produce a better result because the allocation between land disposal and chemical treatment also matters. A constrained optimization method could identify the lowest-cost plan that meets the 20 kg/day limit between YUK, land disposal, and chemical treatment altogether.

Problem 5.6 (4)

Find the strategies which minimize cost and YUK discharge (these will be different strategies) analytically and find the values of the objective metrics. Plot these values in the plot that you created for Problem 4.3. How do their values compare to the spread of values that you found in that problem?

The wastewater allocations satisfy

$$X_1 + X_2 + X_3 = 100, X_1, X_2, X_3 \geq 0.$$

The treatment cost is

$$C = \frac{X_1^2}{20} + 1.5 X_2.$$

Because both terms are nonnegative, cost is minimized when

$$X_1 = 0, X_2 = 0, X_3 = 100.$$

Therefore,

$$C_{min} = 0 \text{ dollars/day}$$

and the corresponding YUK discharge is

$$Y = 0.2(0) + 0.005(0)^2 + 100 = 100 \text{ kg/day}.$$

Minimum Cost: (0,0,100)

To minimize YUK discharge, substitute $X_3 = 100 - X_1 - X_2$ into the discharge equation:

$$Y = 0.2 X_1 + 0.005 X_2^2 + 100 - X_1 - X_2.$$

Thus,

$$Y = 100 - 0.8 X_1 - X_2 + 0.005 X_2^2.$$

For any fixed value of X_2 , increasing X_1 lowers the discharge, so the minimum occurs with no direct discharge:

$$X_3=0, X_1=100 - X_2.$$

Substitution gives

$$Y = 20 - 0.2 X_2 + 0.005 X_2^2.$$

Differentiating with respect to X_2 gives

$$\frac{dY}{dX_2} = -0.2 + 0.01 X_2.$$

Setting this derivative equal to zero gives

$$X_2 = 20.$$

Therefore, the minimum-discharge allocation is

$$X_1 = 80, X_2 = 20, X_3 = 0.$$

Its objective values are

$$Y_{min} = 18 \text{ kg/day}$$

and

$$C = \frac{80^2}{20} + 1.5(20) = 350 \text{ dollars/day}.$$

This is plotted above! (before problem 5.4)

References

List any external references consulted, including classmates.

Jasper Agrawal (ja627) helped me learn how Github works and how to connect a repository to Visual Studio code

Juliet Sencion (jas2224) and I completed the assignment on our own separately and then came together to compare answers and submit it together to Gradescope.

I used ChatGPT to help out on problem 5.3 for code syntax help with graphing arrays. Prompt: "what is wrong with my scatter plot code, why isn't YUK being graphed with colors indicating what is compliant. Also, what is the syntax for graphing a dashed line for 20kg/day" - I also attached photos of my formulas and code to the prompt as context for the AI to use in its answer

Syntax website I used as a resource for help: <https://enccs.github.io/julia-intro/syntax-intro/>

I consulted and referred to previous assignments I completed in CEE 3040 (Uncertainty Analysis in Engineering), which used LaTeX in Visual Studio Code, to ensure that my LaTeX syntax and formatting were correct.

For problem 3.3, I used AI overview for help:

if i want to subtract an integer from every value in an array, how do I do that in Julia

AI Mode

All

Videos

Short videos

Forums

Shopping

Images

More

Tools

AI Overview

To subtract an integer from every value in an array in Julia, **use the dot broadcasting operator** `.-`. The dot tells Julia to apply the subtraction element-wise across the entire array. [The Julia Programming... +2](#)

1. Create a New Array (Element-wise Subtraction)

If you want to leave your original array unchanged and return a new one, use `.-`:

```
julia

# Your starting array
arr = [10, 20, 30, 40]

# Subtract 5 from every element
result = arr .- 5

println(result)
# Output: [5, 15, 25, 35]
```

Use code with caution.

The Julia Pro

How to add/sub

array column?

Aug 24, 2020 —

or array column,

Stack Overflo

Best way to su

Jun 6, 2017 — To

you can use the l

The Julia Pro

Addition/Subtr

- General Usag

Jul 7, 2021 — In J

perform operatio